3 AutoEncoders [30 marks]

3.1 Part 1: Autoencoder for Image Reconstruction [12]

In this part, you will build and train a deep autoencoder from scratch to reconstruct images from the MNIST dataset. You should utilize the MLP components (Linear layers, Activations, Loss functions, Optimizers) that you have already implemented.

3.1.1 Autoencoder Implementation [6]

Create an **MLPAutoencoder** class using your existing MLP framework. This class will define the architecture of your autoencoder.

- **Encoder:** This part of the network will take an input image and compress it into a lower-dimensional latent representation (the "bottleneck").
- **Decoder:** This part will take the latent representation and attempt to reconstruct the original input image.

3.1.2 Training and Visualization on MNIST [6]

- **Training [3]:** Train your MLPAutoencoder on the MNIST training dataset. Your training loop should:
  1. Perform a forward pass to get the reconstructed output.
  2. Calculate the reconstruction loss using Mean Squared Error (MSE).
  3. Perform a backward pass to compute gradients for all parameters.
  4. Update the model's parameters using optimizer of your own choice.
- **Visualization [3]:** After training, visualize the performance of your autoencoder. For each digit class (0-9), display the original image alongside its reconstructed version from the test set.

3.2 Part 2: Anomaly Detection with Autoencoders [18]

Now, you will apply your autoencoder to an anomaly detection task on the **Labeled Faces in the Wild (LFW)** dataset. The core idea is to train the autoencoder to learn what the "normal" data looks like. When presented with "abnormal" or anomalous data, it should produce a significantly higher reconstruction error.

3.2.1 Anomaly Detection [8]

- **Training on Normal Data [3]:** For the "normal" class, we will use the class which has a significant number of images in the LFW dataset; this class is 'George_W_Bush'. This is to ensure sufficient learning of the autoencoder. Train your autoencoder exclusively on images from this normal class. The idea is that the autoencoder will learn

to reconstruct images of this class well but will have a higher reconstruction error for images from other classes ("anomalies").

- **Evaluation [5]:** Evaluate your trained autoencoder on the entire LFW test set. For each image, calculate the reconstruction error (MSE) and use it to classify images as normal or anomalous. You should:

    1. Determine a suitable threshold for the reconstruction error.
    2. Calculate and report the AUC Score, Precision, Recall, and F1-Score.

3.2.2 Analysis and Visualization [10]

- **Bottleneck Dimension Analysis [4]:** Investigate how the bottleneck dimension affects performance.

    1. Choose three different bottleneck dimensions.
    2. For each dimension, train a new autoencoder and evaluate its anomaly detection performance.
    3. Plot the ROC curves for all three models on a single graph and report their AUC scores.

- **Visualization [6]:** Provide a comprehensive visualization of your best model's results.

    1. Correct Classifications: Show an example of a correctly identified "normal" image (True Negative) and a correctly identified "anomaly" (True Positive), displaying the original, the reconstruction, and the reconstruction error for each.
    2. Misclassifications: Show an example of a misclassified "normal" image (False Positive) and a misclassified "anomaly" (False Negative), displaying the original, the reconstruction, and the reconstruction error for each.
    3. PR Curve: Plot the Precision-Recall (PR) curve for your best model.